AMRITA VISHWA VIDYAPEETHAM
DEEMED TO BE UNIVERSITY

Amrita School of AI
COIMBATORE CAMPUS

LABS · EXERCISES · ASSIGNMENTS

Classroom 50: Every Graded Item

One page with the accept link, the marks and the deadline for everything that is graded by tests this semester, and the six commands you need to work an item from accept to report.

Prepared by **Dr. Abhijith Anandakrishnan**
Assistant Professor, Amrita School of AI

SECTIONS
AIM A (aim-a-hpc) and AIM B (aim-b-hpc)

ORGANISATION
Amrita-School-of-AI

TOOL
gh student

UPDATED
15 Oct 2026

OFFERING
Odd Semester 2026-27

Read this first

Every lab, coding exercise and assignment in this course is a **Classroom 50** item: a private GitHub repository created for you the moment you accept it, with an autograder that reports on every submission within a couple of minutes. There is nothing to email and nothing to upload. You accept, you work, you submit, you read the report, you submit again if you want to.

Use the links for **your own section**. Accepting an item in the other section creates a repository nobody will grade.

PITFALL · YOUR GITHUB USERNAME MUST BE THE ONE ON THE ROSTER

You accepted an invitation to the `Amrita-School-of-AI` organisation at your college address. Only that account can accept items. If `gh student whoami` prints a different name, log out and log in with the right account before accepting anything.

Part 1: One-time setup

Do this once, on the machine you will work from (your laptop or the cluster login node). It takes about fifteen minutes if your SSH key already works; see `student-guide.pdf` Part 2 if it does not.

```
# 1. GitHub CLI (Debian/Ubuntu; see cli.github.com for other systems)
sudo apt install gh

# 2. Log in, then add the student tool and log in with the scopes it needs
gh auth login                       # choose GitHub.com, SSH, browser login
gh extension install foundation50/gh-student
gh student login
gh student whoami                   # must print your roster username

# 3. Compiler, OpenMP and MPI for local checks
sudo apt install build-essential openmpi-bin libopenmpi-dev libopenblas-dev
```

On the cluster, `gh` and the compilers are already installed; run only steps 2.

Part 2: Working an item, start to finish

Six commands. The example is Exercise 06 in section A; substitute your section and the item you are working on.

```
# 1. Accept: creates Amrita-School-of-AI/aim-a-hpc-ex06-openmp-reduction-<you>
gh student accept Amrita-School-of-AI aim-a-hpc ex06-openmp-reduction
#    or open the accept link from the tables below in a browser and click Accept

# 2. Clone it (the https:// form asks for a password and fails; use one of these)
gh repo clone Amrita-School-of-AI/aim-a-hpc-ex06-openmp-reduction-<you>
cd aim-a-hpc-ex06-openmp-reduction-<you>

# 3. Read README.md. It is the whole statement, including the marks table.

# 4. Work, and run the public tests as often as you like
make check

# 5. Commit and submit
git add -A
git commit -m "say what you changed"
gh student submit

# 6. Read the report, fix, repeat. Before the second submit:
git pull --rebase origin main
```

Where the report is

About a minute after `gh student submit`:

- the commit on GitHub gets a **green tick or a red cross**;
- a **Release** appears on your repository, named `submit/<time>-<sha>`. Its text is the grader's report: one line per check, PASS or FAIL, the marks for it, and the reason for every failure;
- the same text is under the run's **Summary** in the Actions tab;
- on classroom50.org, **My submission** shows the score and links to the release.

If there is no release after two minutes, open the Actions tab. A red run whose log ends in a grader error, with no failing test named, is a problem on my side: send me the link.

What you may and may not edit

The README of each item names the files that are yours: usually `src/*.c`, sometimes `answers.yaml` or a CSV under `results/`. Everything else is fixed. `.gitignore` and `.github/` are restored on every submit; `tests/` holds copies of the public tests and the grader uses its own copy, so changing them changes nothing. The reference functions marked *given* in a header must stay as they are: the tests compare your versions against them.

How marks are earned

KIND OF CHECK	WHAT PASSES IT
Compiles	`-Wall -Wextra -Werror`. Fails, and nothing after it runs.
Required constructs	the mechanism the item teaches is in the source; comments do not count
Public tests	the tests `make check` ran
Hidden tests	the same behaviour on inputs you have not seen: odd sizes, tiny sizes, awkward rank counts
Determinism	identical output over repeated runs at several thread counts
Speedup	the faster version is faster by a modest factor on a two-core machine; only checked once the code is correct
Analysis	your program computes speedup, efficiency, Karp-Flatt or roofline quantities exactly on hidden data
Committed results	the CSV you produced on the cluster has the right columns, rows and ranges
Answers	numbers in `answers.yaml` against a key, with a tolerance

Some marks cannot be earned on the grading machine. Items marked *GPU cluster* in the tables have those checks run on `asaicompute` after the deadline, and added to your total then; the report tells you how many marks are pending, and an item is scored out of the checks that could run, so a pending cluster check never counts against you. The four assignments carry 4 marks for a short written section, `EXPLANATION.md`, graded against the rubric printed in the README. Everything else is marked by the tests, and of everything you submit the best result counts, after the extension multiplier in Part 3.

Part 3: The items

Accept links are per section. `A` is `aim-a-hpc`, `B` is `aim-b-hpc`. Due times are 23:59 IST on the day shown.

EVERYTHING IS NOW DUE SATURDAY 14 NOVEMBER, 23:59 IST

Every graded item was extended on 2 November to 7 November, and again on 7 November to 14 November. The second extension is because the first one was never announced, so most of the class had no way of knowing about it. Three things went wrong, and every one of them was on our side.

The School's GitHub account ran out of its monthly Actions minutes on **22 October at 21:53 IST**. From that minute until 1 November no submission was graded at all: the Actions run went red within a few seconds and no release appeared, however good the code was. If that happened to you, you did nothing wrong. There are 146 such runs across 117 repositories and I have re-run every one of them, so those submissions are now graded at the time you pushed them and count as on time. You do not need to resubmit for that.

The VPN that reaches `asaicompute` was down for most of that week too, so the parts of an item that need the cluster could not be done. It came back on 2 November. Nobody loses marks for the gap: every item is scored out of the checks that could actually run, and the cluster checks are added once they have been run.

`ex19-cuda-tiled-matmul` scored zero for every submission ever made to it, because the item's own configuration named a test file that does not exist, so its compile step could never succeed and every check behind it was skipped. The item is corrected and every submission has been regraded. Nothing you did caused it and there is nothing to resubmit.

How the extension is weighted, so that submitting early was worth something:

WHEN YOU PUSHED THE SUBMIT TAG	YOUR MARKS ARE MULTIPLIED BY
on or before the item's original deadline	1.00
in the grader outage, 22 Oct 21:53 IST to 1 Nov	1.00
1 Nov to 7 Nov 23:59, while the extension was unannounced	0.90
8 Nov to 14 Nov 23:59	0.80

The 0.80 is the handbook's existing late factor, the same one printed in the previous version of this page; only the 48-hour window has been widened to 14 November. The 0.90 row exists because the 7 November deadline was never announced: anyone who found it in this handbook and submitted against it was working to a deadline the rest of the class did not know about, and that is worth more than work done after the announcement. One of the ten course marks for these items is separately a punctuality mark, awarded on the fraction of items you submitted by their original deadline, with the outage window counted as on time.

Each item keeps the best result of everything you submit, after the multiplier is applied. Improving an item during the extension can raise your mark and cannot lower it, so there is no reason to hold back a better version.

> **PITFALL · RUN `make check` BEFORE EVERY `gh student submit`**
>
> A grading run costs the School about two Actions minutes out of a monthly allowance the whole class shares, and a submit tag that fails to compile spends one for nothing. Build and test locally first, then submit.

The order in the tables is the order the material is taught, and the sensible order to do them in.

> **PITFALL · ONE ITEM EVERY TWO DAYS IS THE PACE, NOT A SUGGESTION**
>
> The dates are close together on purpose. An item left for the weekend before the deadline collides with the two behind it. Do each one the week its material is covered and the schedule is light; leave them and it is not.

Warm-up (no marks)

ITEM	NAME	MARKS	ACCEPT
`a0-git-warmup`	A0: Git and GitHub warm-up	0	A · B

Do A0 first. It is the whole loop above on a task that cannot hurt you, and it tells us both that your account, your tools and the grader agree.

Coding exercises (2% of the course, pooled)

ITEM	NAME	MARKS	DUE (IST)	ACCEPT	GRADED WHERE
`ex01-performance-metrics`	Ex 01: Performance metrics and the roofline	10	Sat 14 Nov, 23:59	A · B	runner only
`ex02-cache-blocking`	Ex 02: Cache blocking	12	Sat 14 Nov, 23:59	A · B	runner only
`ex06-openmp-reduction`	Ex 06: Parallel reduction with OpenMP	10	Sat 14 Nov, 23:59	A · B	runner only
`ex07-openmp-load-balance`	Ex 07: Load imbalance and the schedule clause	12	Sat 14 Nov, 23:59	A · B	runner only
`ex08-openmp-histogram`	Ex 08: Parallel histogram	12	Sat 14 Nov, 23:59	A · B	runner only
`ex11-mpi-halo-stencil`	Ex 11: MPI halo exchange	10	Sat 14 Nov, 23:59	A · B	runner only
`ex12-mpi-collectives`	Ex 12: MPI collectives	11	Sat 14 Nov, 23:59	A · B	runner only
`ex17-cuda-vector-add`	Ex 17: First CUDA kernels	10	Sat 14 Nov, 23:59	A · B	7 on the GPU cluster
`ex19-cuda-tiled-matmul`	Ex 19: Shared-memory tiling	12	Sat 14 Nov, 23:59	A · B	7 on the GPU cluster
`ex20-cuda-reduction`	Ex 20: Parallel reduction on a GPU	10	Sat 14 Nov, 23:59	A · B	7 on the GPU cluster

Each exercise is one file in `src/`. The starters compile and are correct serial code; the marks are for the parallel or optimised version the statement asks for, and for keeping it correct on the hidden inputs.

Lab sheets (2% of the course, pooled)

ITEM	NAME	MARKS	DUE (IST)	ACCEPT	GRADED WHERE
`lab01-timing-and-memory`	Lab 1: Timing and memory	10	Sat 14 Nov, 23:59	A · B	runner only
`lab02-slurm-and-the-cluster`	Lab 2: SLURM and the cluster	10	Sat 14 Nov, 23:59	A · B	runner only
`lab03-scaling-study-and-roofline`	Lab 3: Scaling study and roofline	10	Sat 14 Nov, 23:59	A · B	runner only
`lab04-openmp-and-races`	Lab 4: OpenMP and races	10	Sat 14 Nov, 23:59	A · B	runner only
`lab05-optimising-matrix-multiply`	Lab 5: Optimising matrix multiply	10	Sat 14 Nov, 23:59	A · B	runner only
`lab06-mpi-basics-and-deadlock`	Lab 6: MPI basics and deadlock	10	Sat 14 Nov, 23:59	A · B	runner only
`lab07-domain-decomposition`	Lab 7: Domain decomposition	10	Sat 14 Nov, 23:59	A · B	runner only
`lab08-first-cuda-kernels`	Lab 8: First CUDA kernels	10	Sat 14 Nov, 23:59	A · B	3 on the GPU cluster
`lab09-cuda-profiling-and-roofline`	Lab 9: CUDA profiling and roofline	10	Sat 14 Nov, 23:59	A · B	runner only

The lab sheets keep their original teaching text; the last section of each README, *What is graded*, says which deliverables the tests look at. Measurements that need the cluster are committed as CSVs and shape-checked; the numbers themselves are yours to interpret.

Assignments (6% of the course, pooled)

ITEM	NAME	MARKS	DUE (IST)	ACCEPT	GRADED WHERE
`a1-measure-before-you-optimise`	Assignment 1: Measure before you optimise	30	Sat 14 Nov, 23:59	A · B	4 by rubric
`a2-openmp-scaling`	Assignment 2: OpenMP scaling	30	Sat 14 Nov, 23:59	A · B	4 by rubric
`a3-mpi-decomposition`	Assignment 3: MPI decomposition	30	Sat 14 Nov, 23:59	A · B	4 by rubric
`a4-gpu-and-ai`	Assignment 4: GPU and AI	30	Sat 14 Nov, 23:59	A · B	14 on the GPU cluster; 4 by rubric

Thirty marks each: 26 from tests, 4 from `EXPLANATION.md` (at most 250 words, rubric in the README). The kernel is fixed for everyone so that the tests are the same for everyone: Jacobi stencil, CSR sparse matrix-vector product, 2-D heat diffusion with MPI, and a CUDA matrix transpose.

Part 4: The GPU cluster

Eleven items need a GPU or a real multi-core machine: Labs 2, 3, 8 and 9, Exercises 17, 19 and 20, and all four assignments. You do not get your own cluster account; the class shares one, `hpccourse`, on `asaicompute`.

Getting in

Access is by **your own GitHub SSH key**, the same one you use to push. Nothing to request and no password: if your key is on your GitHub account, it already works.

```
ssh hpccourse@172.17.16.11
```

You land in your own directory, `w/<your roll number>`, and the prompt tells you so. If you are refused, your GitHub account has no SSH key on it; add one at github.com/settings/keys and tell me so I can refresh the list.

> **NOTE · YOU NEED THE AMRITA VPN TO REACH THE CLUSTER**
>
> The student wifi range is blocked from reaching the servers, so the VPN is the only way in from a laptop. Connect to it first, then `ssh`. It was deactivated for most of the last week of October, which is why nobody could get in; it was restored on 2 November. If `ssh` is refused while the VPN is up, the reason is the missing key above.

Copying work across

`rsync` and `scp` both work. Clone your repository on your laptop as usual, then push the files up:

```
rsync -av ./src/ hpccourse@172.17.16.11:src/
scp results.csv hpccourse@172.17.16.11:
```

Pull results back the same way, and commit them from your laptop. Do not try to run `git` on the cluster; the account is deliberately limited to building and running.

Running something

Two commands, and only these two. They set the partition, the account and the limits for you:

```
course-run make check            # run it now, on a GPU
course-submit scaling.sbatch     # queue a batch script
course-status                    # see the queue
```

Both cap a job at **five minutes** and one GPU. That is ample: these assignments run in one or two minutes. Nothing else needs a flag. A plain `sbatch` or `srun` is **refused** with `Job violates accounting/QOS policy`: the course account may only use the course queue, and these three commands select it for you. Asking for more than five minutes is refused the same way.

You may submit **at any time of day**. The course queue has its own five-minute limit, so a student job never holds the machine for long.

Check on your jobs with **`course-status`**, and stop one with `scancel <jobid>`. Use `course-status` rather than plain `squeue`: the course partition is hidden from the cluster-wide view, so `squeue` on its own will not list these jobs. Everybody's jobs appear there; yours carry your roll number as the job name.

A job waiting with reason `(Resources)` or `(Priority)` is simply queued behind work already on the GPUs. It starts by itself when a card frees up, with nothing more from you. Research jobs on this machine can run for days, so a card is not always free the minute you ask: submit the evening before a deadline rather than the hour before it.

Share the space sensibly

It is one account, so **everyone can see everyone's directories**. Keep nothing private there, and copy nothing out of anybody else's. Submissions are compared for similarity, and marks come from what you push to your own repository, so copying buys you nothing and costs you a great deal.

Part 5: When something goes wrong

SYMPTOM	WHAT IT MEANS
`gh student accept` says the assignment is not found	You are in the other section, or the item is not released yet.
`git clone https://...` asks for a password	Use `gh repo clone` or the `git@github.com:` address.
`make check` passes, the report shows hidden tests failing	Your code is right for the sizes you tried. Read the failing test's name: it names the case.
`! [rejected] main -> main (fetch first)` on push	Submitting added a commit. `git pull --rebase origin main`, then push again.
Speedup check says *skipped: ... failed*	Speed is only measured once correctness passes. Fix the tests first.
The report says marks are graded on the cluster	Normal for CUDA items. They arrive after the deadline.
No release two minutes after submitting	Open the Actions tab. If the run itself is red, send me the link.
`nvcc: command not found` on your laptop	CUDA items build on the cluster (Part 4), not on a laptop without a GPU.
`ssh hpccourse@...` refused	Connect to the Amrita VPN first. If it is already up, your GitHub account has no SSH key on it.
Job pending at `(Resources)`	The GPUs are busy with research jobs. It starts on its own; do not resubmit.
`Requested reservation not usable now`	Same thing: you are outside the window.